```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve,
    precision_recall_curve, f1_score
)

%matplotlib inline

train = pd.read_csv('../data/processed/training_lr_mlp.csv')
test  = pd.read_csv('../data/processed/test_lr_mlp.csv')

X_train = train.drop(columns=['label'])
y_train = train['label']
X_test  = test.drop(columns=['label'])
y_test  = test['label']

print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(f'Class balance (train) — 0: {(y_train==0).sum()}, 1:
{(y_train==1).sum()}')
```

```
Train: (107740, 58), Test: (55945, 58)
Class balance (train) — 0: 51890, 1: 55850
```

```
from sklearn.neural_network import MLPClassifier

base_model = MLPClassifier(
    random_state=42,
    max_iter=300,
    early_stopping=True
)
```

## Hyperparameter Tuning

```
from sklearn.model_selection import RandomizedSearchCV
param_dist = {
    'hidden_layer_sizes': [(32,), (64,), (128,), (256,), (64, 32),
(128, 64), (128, 32), (256, 128), (256, 32), (128, 64, 32), (256, 128,
64), (256, 128, 64, 32)],
    'activation': ['relu', 'tanh'],
    'learning_rate_init': [1e-2, 1e-3, 5e-4, 3e-4, 1e-4],
    'alpha': [1e-4, 1e-3, 1e-2, 0.1, 1.0],
    'batch_size': [32, 64, 128, 256],
}

search = RandomizedSearchCV(
    base_model,
```

```
    param_distributions=param_dist,
    n_iter=50,
    scoring='roc_auc',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

search.fit(X_train, y_train)

print(f'Best ROC-AUC (CV): {search.best_score_:.4f}')
print(f'Best params:       {search.best_params_}')
```

```
Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best ROC-AUC (CV): 0.9884
Best params:       {'learning_rate_init': 0.01, 'hidden_layer_sizes':
(128, 64), 'batch_size': 256, 'alpha': 0.001, 'activation': 'relu'}
```

```
# hard-coding the optimal parameters from previous hyperparameter run
to prevent running it again.
optimal_params =  {'learning_rate_init': 0.01, 'hidden_layer_sizes':
(128, 64), 'batch_size': 256, 'alpha': 0.001, 'activation': 'relu'}

#model = search.best_estimator_
model = MLPClassifier(**optimal_params, random_state=42, max_iter=300,
early_stopping=True)

model.fit(X_train, y_train)
```

```
MLPClassifier(alpha=0.001, batch_size=256, early_stopping=True,
              hidden_layer_sizes=(128, 64), learning_rate_init=0.01,
              max_iter=300, random_state=42)
```

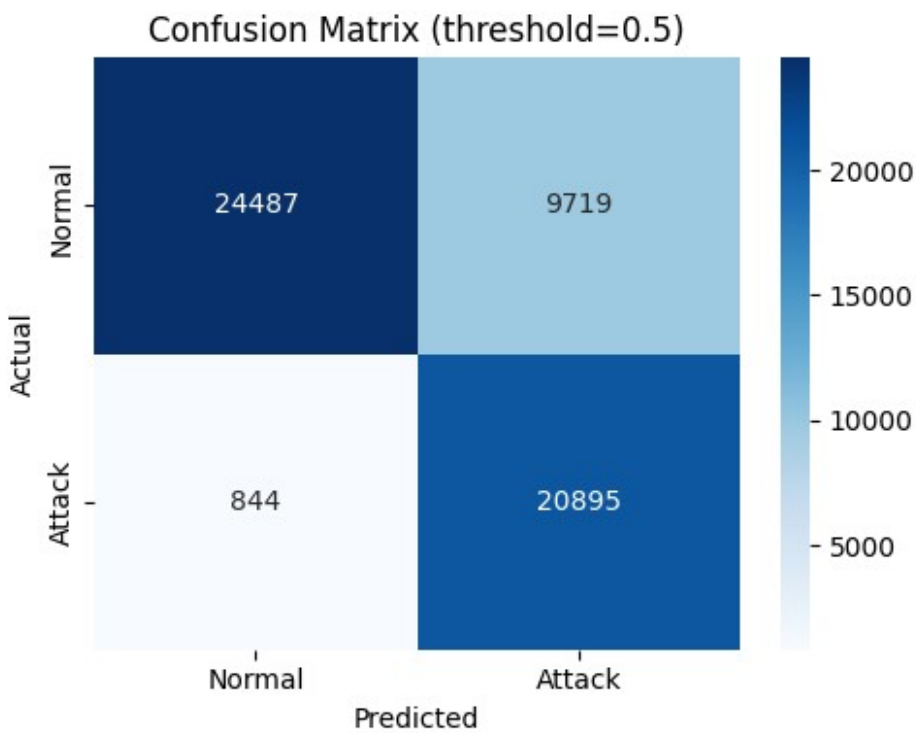
## Base Evaluation, threshold = 0.5

```
y_prob = model.predict_proba(X_test)[:, 1]
y_pred = (y_prob >= 0.5).astype(int)

print(classification_report(y_test, y_pred, target_names=['Normal',
'Attack']))
print(f'ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}')
```

```
              precision    recall  f1-score   support

      Normal       0.97      0.72      0.82     34206
      Attack       0.68      0.96      0.80     21739

    accuracy                           0.81     55945
   macro avg       0.82      0.84      0.81     55945
weighted avg       0.86      0.81      0.81     55945
```

```
ROC-AUC: 0.9578
```

```
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['Normal', 'Attack'], yticklabels=['Normal',
'Attack'])
ax.set_ylabel('Actual')
ax.set_xlabel('Predicted')
ax.set_title('Confusion Matrix (threshold=0.5)')
plt.tight_layout()
plt.show()
```



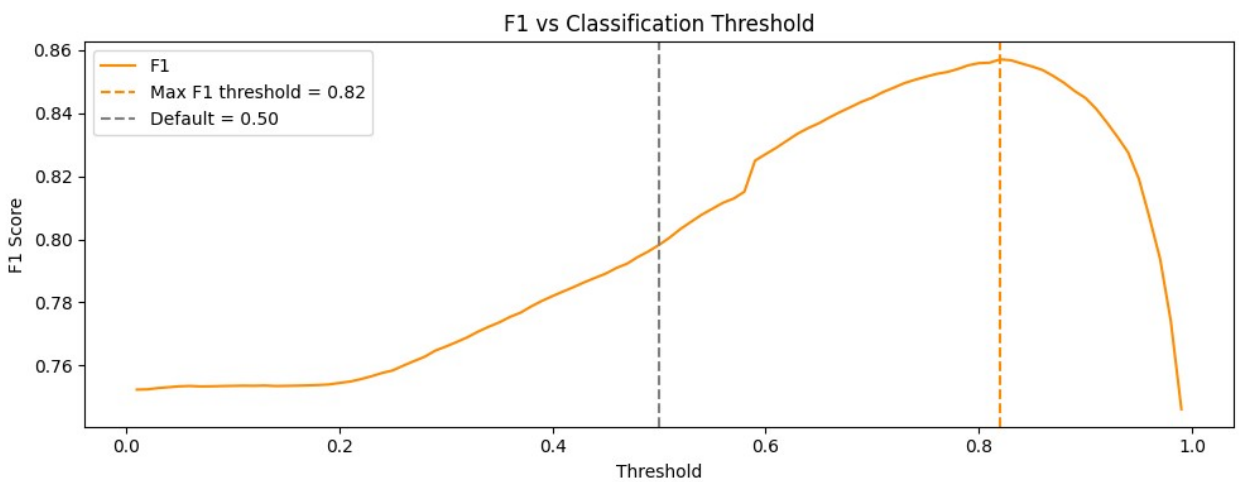

## Threshold Tuning

In cybersecurity, false negatives (missed attacks) are more costly than false positives. We sweep thresholds from 0.01 to 0.99 and choose the threshold that maximises F1.

```
thresholds = np.arange(0.01, 1.0, 0.01)
f1_scores  = [f1_score(y_test, (y_prob >= t).astype(int)) for t in
thresholds]

best_f1_threshold = thresholds[np.argmax(f1_scores)]

print(f'Optimal threshold (max F1): {best_f1_threshold:.2f}  — F1:
```

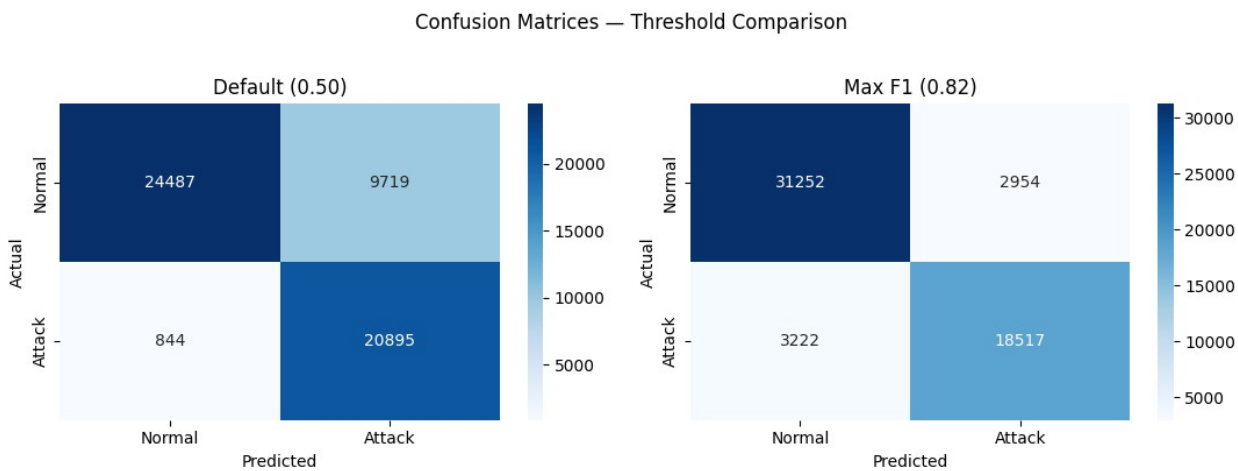
```
{max(f1_scores):.4f}')
print(f'Default threshold (0.5):           — F1: {f1_scores[49]:.4f}')

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(thresholds, f1_scores, color='darkorange', label='F1')
ax.axvline(best_f1_threshold, color='darkorange', linestyle='--',
label=f'Max F1 threshold = {best_f1_threshold:.2f}')
ax.axvline(0.5, color='gray', linestyle='--', label='Default = 0.50')
ax.set_xlabel('Threshold')
ax.set_ylabel('F1 Score')
ax.set_title('F1 vs Classification Threshold')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Optimal threshold (max F1): 0.82  — F1: 0.8571
Default threshold (0.5):           — F1: 0.7982
```




```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (t, title) in zip(axes, [
    (0.5,                 'Default (0.50)'),
    (best_f1_threshold, f'Max F1 ({best_f1_threshold:.2f})'),
]):
    preds = (y_prob >= t).astype(int)
    sns.heatmap(confusion_matrix(y_test, preds), annot=True, fmt='d',
cmap='Blues', ax=ax,
                xticklabels=['Normal', 'Attack'],
yticklabels=['Normal', 'Attack'])
    ax.set_ylabel('Actual')
    ax.set_xlabel('Predicted')
    ax.set_title(title)
plt.suptitle('Confusion Matrices — Threshold Comparison', y=1.02)
plt.tight_layout()
plt.show()
```

```
print(f'\nChosen threshold: {best_f1_threshold:.2f} (max F1)')
print(classification_report(y_test, (y_prob >=
best_f1_threshold).astype(int), target_names=['Normal', 'Attack']))
```




```
Chosen threshold: 0.82 (max F1)
              precision    recall  f1-score   support

      Normal       0.91      0.91      0.91     34206
      Attack       0.86      0.85      0.86     21739

    accuracy                           0.89     55945
   macro avg       0.88      0.88      0.88     55945
weighted avg       0.89      0.89      0.89     55945


y_pred_tuned = (y_prob >= best_f1_threshold).astype(int)
```

# Per-Attack-Category Breakdown

Join predictions against the raw test data to evaluate performance per attack type.

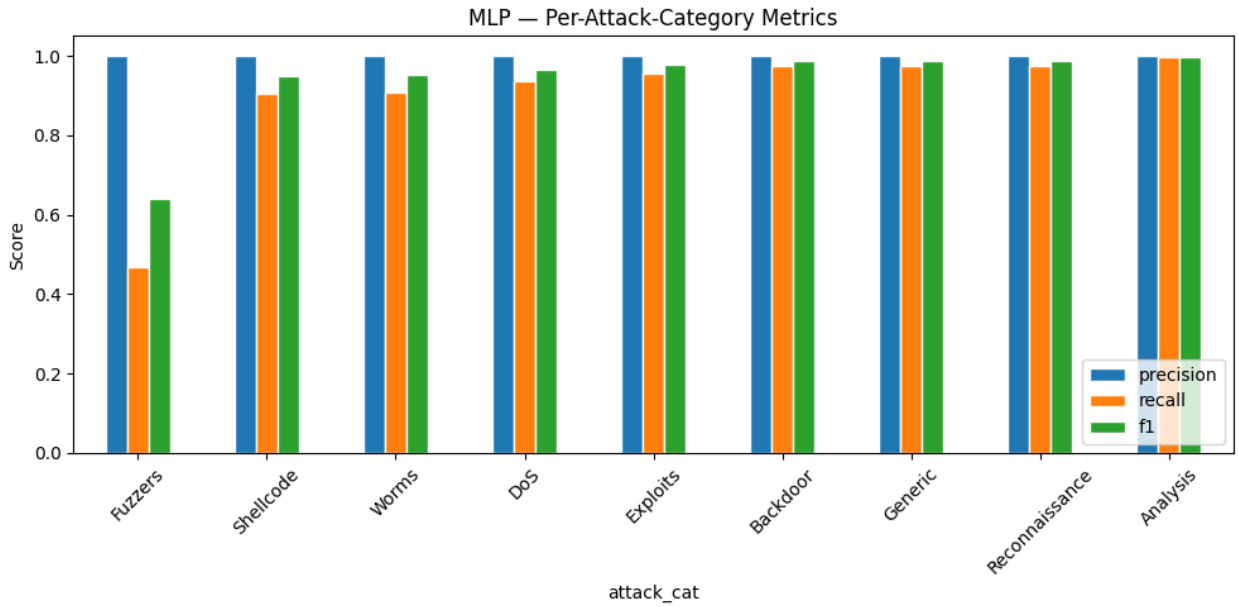
```
raw_test = pd.read_csv('../data/raw/UNSW_NB15_testing-set.csv')

raw_test = raw_test.drop_duplicates(subset=[c for c in
raw_test.columns if c != 'id']).reset_index(drop=True)

results = raw_test[['attack_cat']].copy()
results['y_true'] = y_test.values
results['y_pred'] = y_pred_tuned
results['y_prob'] = y_prob

results.head()
```

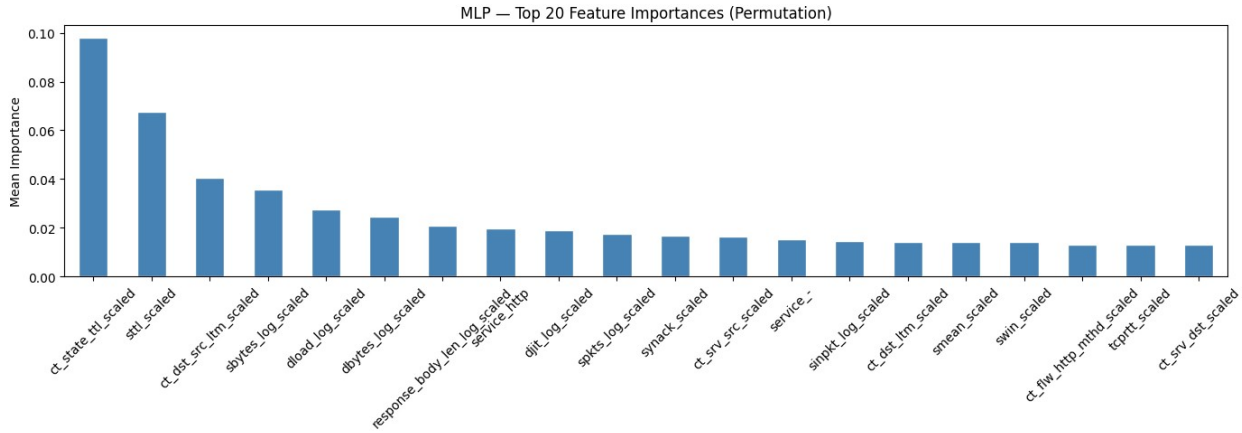
```
  attack_cat  y_true  y_pred    y_prob
0     Normal       0       0  0.816330
1     Normal       0       1  0.833462
2     Normal       0       0  0.810413
3     Normal       0       0  0.743074
4     Normal       0       0  0.767552
```

```
def attack_metrics(group):
    tp = ((group['y_true'] == 1) & (group['y_pred'] == 1)).sum()
    fn = ((group['y_true'] == 1) & (group['y_pred'] == 0)).sum()
    fp = ((group['y_true'] == 0) & (group['y_pred'] == 1)).sum()
    tn = ((group['y_true'] == 0) & (group['y_pred'] == 0)).sum()
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall    = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1        = 2 * precision * recall / (precision + recall) if
(precision + recall) > 0 else 0
    return pd.Series({'count': len(group), 'precision':
round(precision, 4),
                      'recall': round(recall, 4), 'f1': round(f1, 4)})

attack_results = results[results['attack_cat'] != 'Normal']
breakdown = attack_results.groupby('attack_cat').apply(attack_metrics,
include_groups=False).sort_values('recall')
print(breakdown.to_string())
```

```
                  count  precision  recall      f1
attack_cat
Fuzzers          4838.0        1.0  0.4694  0.6389
Shellcode         378.0        1.0  0.9048  0.9500
Worms              44.0        1.0  0.9091  0.9524
DoS              1718.0        1.0  0.9354  0.9666
Exploits         7609.0        1.0  0.9564  0.9777
Backdoor          346.0        1.0  0.9740  0.9868
Generic          3657.0        1.0  0.9746  0.9871
Reconnaissance   2703.0        1.0  0.9748  0.9873
Analysis          446.0        1.0  0.9955  0.9978
```

```
fig, ax = plt.subplots(figsize=(10, 5))
breakdown[['precision', 'recall', 'f1']].plot(kind='bar', ax=ax,
edgecolor='white')
ax.set_title('MLP — Per-Attack-Category Metrics')
ax.set_ylabel('Score')
ax.set_ylim(0, 1.05)
ax.tick_params(axis='x', rotation=45)
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```




# Feature Importance

```
from sklearn.inspection import permutation_importance

result = permutation_importance(model, X_test, y_test, n_repeats=10,
random_state=42, n_jobs=-1)
importances = pd.Series(result.importances_mean,
index=X_train.columns).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(14, 5))
importances.head(20).plot(kind='bar', ax=ax, color='steelblue',
edgecolor='white')
ax.set_title('MLP — Top 20 Feature Importances (Permutation)')
ax.set_ylabel('Mean Importance')
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```